Bureau Wbtv Registry Tracker

Bureau Wbtv Registry Tracker is a Python project for collecting interpreter and translator registry data and monitoring changes in the Bureau Wbtv public registry. Its bulk fetcher uses HTTP requests directly; it does not automate a browser. A separate monitor can periodically check for new registrations and optionally send Telegram alerts and email notificaton.

Features

- Search the target languages listed in `target_lang.json` (430 entries in the current list).
- Search for interpreters (`Tolk`), translators (`Vertaler`), or both.
- Fetch result records and, by default, enrich them with available profile contact and registration details.
- Save structured JSON and an Excel workbook; optionally export CSV.
- Filter a run to one language, a name substring, a GUID, or a limited number of languages.
- Monitor for newly seen language/role registrations, keep local tracking files in sync, and optionally notify a Telegram chat.
- Run unit tests with mocked HTTP interactions; the test suite does not need live access to the portal.

Requirements

- Windows PowerShell (commands below use PowerShell syntax).
- Python 3.14 or newer.
- `uv` for dependency and project management.
- Network access to the Bureau Wbtv portal for live fetches.
- Telegram bot credentials only if Telegram notifications are desired.

The dependencies are declared in `pyproject.toml` and pinned in `uv.lock`.

Setup

From the project directory:

```
uv sync
```

Tests

The test suite uses mocked HTTP interactions and temporary files; it does not require a live portal connection or send Telegram messages.

```
# Run the complete suite
uv run -m unittest test_fetch_all_languages.py -v

# Run one test method
uv run -m unittest
```

```
test_fetch_all_languages.TestClientSearchWorkflow.test_client_search_language_succ
ess -v
```

Focused test classes:

```
# Request payload construction
uv run -m unittest test_fetch_all_languages.TestPayloadConstruction -v

# Response parsing and validation
uv run -m unittest test_fetch_all_languages.TestResponseParsing -v

# Target-language loading and filtering
uv run -m unittest test_fetch_all_languages.TestLanguageFilteringAndLoading -v

# Result persistence and CSV / Excel exports
uv run -m unittest test_fetch_all_languages.TestResultOrganizationAndPersistence -
v

# Mocked HTTP client workflow
uv run -m unittest test_fetch_all_languages.TestClientSearchWorkflow -v

# Logging setup
uv run -m unittest test_fetch_all_languages.TestLoggingSetup -v
```

Bulk fetch

Try one language first

Run a small query and save it to separate files so the default full-result files are left untouched:

```
uv run fetch_all_languages.py --language "Arabic Egypt" --output
arabic_egypt_test.json
```

`--language` accepts a case-insensitive substring or an exact language GUID. Here are examples using a name substring, GUID, and both profession roles:

```
# Search every target language whose name contains "arabic"
uv run fetch_all_languages.py --language arabic --output arabic_results.json

# Search by language GUID
uv run fetch_all_languages.py --language 46161896180548020 --output
guid_results.json

# Search one language for interpreters and translators
uv run fetch_all_languages.py --language "Arabic Egypt" --both --output
arabic_both.json
```

Fetch the full target-language list

```
uv run fetch_all_languages.py
```

By default, this searches `Tolk` (interpreters) for every entry in `target_lang.json`, fetches profile details, and writes:

- `all_languages_results.json`
- `all_languages_results.xlsx`
- `fetch_all_languages.log`
- `details_cache.json` (cached profile details)

The default JSON and Excel output names are reused on later runs and can be overwritten. Choose custom output paths when you want to preserve existing results.

Search translators or both roles

```
# Translators only
uv run fetch_all_languages.py --roles Vertaler --output translators.json

# Interpreters and translators
uv run fetch_all_languages.py --both --output all_professionals.json
```

The `--roles` option also accepts explicit roles:

```
uv run fetch_all_languages.py --roles Tolk Vertaler --limit 5 --output sample.json
```

Fast runs and bounded samples

Use `--no-details` to skip profile-modal requests for contact information, registrations, and oath dates. This is useful for faster basic-result runs:

```
# First five languages, basic registry fields only
uv run fetch_all_languages.py --no-details --limit 5 --output fast_test.json

# All languages for both roles, without profile enrichment
uv run fetch_all_languages.py --both --no-details --output all_basic.json --delay
0.5
```

Use `--limit` to restrict a run to the first N languages in the selected list:

```
# First three languages, with a half-second delay
uv run fetch_all_languages.py --limit 3 --output first_3.json --delay 0.5
```

```
# First ten languages, both roles, and an additional CSV export
uv run fetch_all_languages.py --limit 10 --both --output first_10.json --csv-
output first_10.csv
```

Full-run recipes

```
# All target languages, default role and pacing
uv run fetch_all_languages.py

# Both roles with explicitly named outputs
uv run fetch_all_languages.py --both --output all_professionals.json --excel-
output all_professionals.xlsx --delay 1.0

# More conservative pacing and a custom log
uv run fetch_all_languages.py --output all_languages_results.json --delay 1.5 --
log-file full_run.log

# Debug-level logging
uv run fetch_all_languages.py --output debug_results.json --delay 1.0 --verbose --
log-file debug_run.log
```

Network settings and alternate language lists

For a small test, you can use a shorter delay and timeout. For slower or unstable connections, increase the delay, timeout, and bounded retry count:

```
# Small sample with shorter timeout
uv run fetch_all_languages.py --limit 5 --delay 0.2 --timeout 15.0

# More conservative pacing and retry allowance
uv run fetch_all_languages.py --delay 2.0 --retries 5 --timeout 60.0

# Use a different language list (same JSON entry shape as target_lang.json)
uv run fetch_all_languages.py --target-file custom_languages.json --output
custom_results.json
```

Complete fetch command-line reference

Option	Default	Purpose
`-h`, `--help`	—	Show help and exit.
`-l`, `--language TEXT`	All languages	Filter by a case-insensitive language-name substring or exact GUID.
`-n`, `--limit N`	No limit	Process at most `N` selected languages.

Option	Default	Purpose
`-o, --output PATH`	`all_languages_results.json`	JSON results output path.
`--excel-output PATH`	JSON path with `.xlsx` suffix	Excel output path.
`--export-excel JSON_FILE`	Disabled	Convert an existing results JSON file to Excel without fetching.
`--csv-output PATH`	Disabled	Also write a CSV export to this path.
`--export-csv JSON_FILE`	Disabled	Convert an existing results JSON file to CSV without fetching. If `--csv-output` is omitted, the CSV path uses the `--output` name with a `.csv` suffix.
`--no-details`	Off	Skip profile-detail enrichment; fetch basic registry fields only.
`--log-file PATH`	`fetch_all_languages.log`	Log output path.
`-t, --target-file PATH`	`target_lang.json`	Target-language JSON file. Each entry must contain `guid` and `name`.
`-d, --delay SECONDS`	`1.0`	Pause between language searches.
`--retries N`	`3`	Maximum retries for transient HTTP failures.
`--timeout SECONDS`	`30.0`	HTTP request timeout in seconds.
`--roles ROLE [ROLE ...]`	`Tolk`	Select `Tolk`, `Vertaler`, `both`, or multiple roles such as `Tolk Vertaler`.
`--both`	Off	Search both `Tolk` and `Vertaler` roles.
`-c, --concurrency N`	`10`	Worker count for profile-detail enrichment.
`-v, --verbose`	Off	Enable debug-level logging.

Relative paths are resolved from the project directory. The default results file names can be overwritten by a later run; use custom paths to keep separate outputs.

See the complete current option list with:

```
uv run fetch_all_languages.py --help
```

Export existing results without fetching

Convert an existing results JSON file to Excel or CSV without making portal requests:

```
uv run fetch_all_languages.py --export-excel all_languages_results.json --excel-output report.xlsx
uv run fetch_all_languages.py --export-csv all_languages_results.json --csv-output report.csv
```

CSV export uses UTF-8 with a byte-order mark (`utf-8-sig`) to improve compatibility with spreadsheet applications.

The fetcher protects reserved project files from being selected as output targets. These include `results.json`, `requests.json`, `cookies.json`, `target_lang.json`, `fetch_all_languages.py`, `test_fetch_all_languages.py`, `monitor.py`, `known_records.json`, `pyproject.toml`, and `uv.lock`.

Continuous monitoring

`monitor.py` checks the selected languages repeatedly (once per hour by default). It compares registrations using a key based on professional GUID, role, and target language. When it finds new registrations, it enriches their details, updates the local result files, and sends a **formatted Email alert by default** (with rich HTML cards and plain-text fallback). Optionally, it can also send a **Telegram notification** if Telegram bot credentials are configured. If neither is configured, alerts are printed to the console.

Configure Email (Default) and Telegram (Optional)

Copy the example configuration:

```
Copy-Item .env.example .env
```

Edit `.env` and set your credentials:

```
# Default Email Alerts (Gmail SMTP / App Password)
SENDER_EMAIL=your_email@gmail.com
EMAIL_PASSWORD=your_16_character_app_password
RECIPIENT_EMAIL=recipient@example.com

# Optional Telegram Bot Alerts
TELEGRAM_BOT_TOKEN=your_bot_token
TELEGRAM_CHAT_ID=your_chat_id

# Monitoring interval in seconds (3600 seconds = 1 hour)
MONITOR_INTERVAL_SECONDS=3600
```

> [!TIP] For Gmail, use an **App Password** (Google Account > Security > 2-Step Verification > App passwords) rather than your main account password.

Keep `.env` private. Do not commit it or share its contents.

Test Notifications

Test your alert connections before starting the monitor:

```
# Test default Email notification
uv run monitor.py --test-email

# Test optional Telegram notification
uv run monitor.py --test-telegram

# Or test email directly via notification.py
uv run notification.py
```

Run the monitor

```
# One check cycle
uv run monitor.py --once

# Continuously, using the default one-hour interval
uv run monitor.py

# Monitor only languages matching a substring
uv run monitor.py --language Arabic

# One cycle for a small sample
uv run monitor.py --once --limit 3

# Both interpreters and translators, every 30 minutes
uv run monitor.py --roles both --interval 1800
```

Stop a continuous run with `Ctrl+C`.

Monitor run recipes

```
# Test Email credentials and delivery (Default)
uv run monitor.py --test-email

# Test Telegram credentials and delivery (Optional)
uv run monitor.py --test-telegram

# Run one monitoring cycle and exit
uv run monitor.py --once

# Run continuously using the configured/default one-hour interval
uv run monitor.py
```

```
# Send alerts to a specific recipient address
uv run monitor.py --recipient-email alerts@example.com

# Disable email alerts and use console/telegram only
uv run monitor.py --disable-email

# Monitor languages whose names contain "Arabic"
uv run monitor.py --language Arabic

# Check only the first three languages once
uv run monitor.py --once --limit 3

# Monitor both roles every 30 minutes
uv run monitor.py --roles both --interval 1800

# Scan a small set of languages using two scan workers
uv run monitor.py --once --limit 5 --scan-workers 2
```

Monitor files and complete command-line reference

By default, the monitor reads `target_lang.json` and uses:

- `known_records.json` to track known registrations.
- `all_languages_results.json` as the baseline and incrementally updated JSON results.
- `all_languages_results.xlsx` as the incrementally updated workbook.
- `monitor.log` for execution logs.

These defaults refer to files in the project directory. Relative paths passed to the monitor are also resolved from the project directory.

Option	Default	Purpose
`-h`, `--help`	—	Show help and exit.
`--recipient-email EMAIL`	`.env` / `RECIPIENT_EMAIL`	Recipient email address(es) for default notifications.
`--sender-email EMAIL`	`.env` / `SENDER_EMAIL`	Sender email address for default notifications.
`--email-password PWD`	`.env` / `EMAIL_PASSWORD`	App password for sender email account.
`--disable-email`, `--no-email`	Off	Disable default email notifications.
`--test-email`	Off	Send a test Email notification and exit.
`--token TOKEN`	`.env` / `TELEGRAM_BOT_TOKEN`	Optional Telegram bot token.
`--chat-id CHAT_ID`	`.env` / `TELEGRAM_CHAT_ID`	Optional Telegram chat ID.

Option	Default	Purpose
`--test-telegram`	Off	Send a test Telegram notification and exit.
`--interval SECONDS`	`3600` or `.env MONITOR_INTERVAL_SECONDS`	Seconds between continuous cycles.
`-l`, `--language TEXT`	All languages	Filter by a case-insensitive language-name substring or exact GUID.
`-n`, `--limit N`	No limit	Check at most `N` selected languages per cycle.
`--roles {Tolk,Vertaler,both} [...]`	`Tolk`	Select one or more profession roles.
`-w`, `--scan-workers N`	`8`	Number of worker threads scanning languages in parallel.
`-c`, `--concurrency N`	`10`	Parallelism for fetching profile details of new records.
`--once`	Off	Run one check cycle and exit.
`--known-file PATH`	`known_records.json`	Registration tracking file.
`--baseline-file PATH`	`all_languages_results.json`	Existing results JSON used to seed the initial baseline.
`--output PATH`	`all_languages_results.json`	JSON results file to update when new registrations are found.
`--excel-output PATH`	`all_languages_results.xlsx`	Excel workbook to update when new registrations are found.
`--log-file PATH`	`monitor.log`	Log output path.
`-v`, `--verbose`	Off	Enable debug-level logging.

See all monitor options with:

```
uv run monitor.py --help
```

Input and output formats

Target language list

`target_lang.json` is a JSON array. Each entry must contain a `guid` and a `name`; the current file also includes `number_of_records`.

```
[
  {
    "guid": "language-id",
    "name": "Language name",
    "number_of_records": 0
  }
]
```

Bulk results JSON

The bulk fetcher writes a JSON object with `metadata` and `results`. Each language result includes its name, GUID, status, count, count breakdown, pagination indicator, and records (or an error when the search fails). Successful records retain the portal's original attributes and are tagged with their role and level. When profile enrichment is enabled, records may also include a `details` object.

The workbook and CSV contain normalized English columns:

`GUID`, `Name`, `Wbtv Number`, `City`, `Role`, `Level`, `Language`, `Telephone 1`, `Telephone 2`, `Email`, `Website`, `Registered Languages`, `Registrations`, and `Date of Oath`.

Unavailable profile fields may be represented by `-`; `--no-details` leaves the optional detail fields empty or at their fallback values.

Tests

Run the unit tests from the project directory:

```
uv run -m unittest test_fetch_all_languages.py -v
```

The tests cover request-payload construction, response parsing, language loading and filtering, result exports and persistence, mocked client behavior, and logging. They do not perform live portal searches or send Telegram messages.

Privacy and responsible use

The registry may return personal information, including contact details. Result JSON, Excel/CSV exports, detail caches, logs, and Telegram alerts may therefore contain sensitive personal data. Store them securely, share them only when appropriate, and avoid committing generated data or `.env` files to source control.

The scripts use the portal's current runtime request flow, which is not documented as a stable public API. Its behavior or payload format may change. Keep request pacing conservative, use the retry and timeout options responsibly, and follow the portal's terms and applicable privacy requirements.